САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа № 4

Развёртывание приложения на удалённом сервере

Выполнил:

Губанов Егор

Группа БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Подготовить удалённый сервер, установить необходимые зависимости, развернуть приложение из лабораторной работы № 3 и настроить nginx как обратный прокси для доступа к API извне. Зафиксировать основные этапы настройки скриншотами.

Ход работы

1 Проверка исходного проекта и сервера

Для работы я использовал сервис аренды недвижимости из каталога labs/lab3. Его docker-compose.yml ссылается на исходный код из labs/lab2. Приложение включает gateway, сервисы auth, catalog, deals и messaging, четыре базы PostgreSQL и RabbitMQ. Код микросервисов при ручном развёртывании не изменялся.

К серверу 77.239.127.10 я подключился по SSH в Termius. На сервере установлена Ubuntu 20.04.6 LTS, Docker Engine 28.1.1 и Docker Compose 2.35.1

```
root@vm22690:~# lsb_release -ds
Ubuntu 20.04.6 LTS
root@vm22690:~# free -h
               total        used        free      shared  buff/cache   available
Mem:           1.9Gi       1.1Gi       109Mi        29Mi       770Mi       667Mi
Swap:          4.0Gi       780Mi       3.2Gi
root@vm22690:~# df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda2        20G   19G  356M  99% /
root@vm22690:~# docker --version
Docker version 28.1.1, build 4eba377
root@vm22690:~# docker compose version
Docker Compose version v2.35.1
```

Рисунок 1. Проверка ОС, памяти, диска и версий Docker

2 Подготовка дискового пространства

Освободил свободное место увеличилось до 2 ГБ. Данные приложений и Docker-тома не удалялись.

```
journalctl --disk-usage
journalctl --vacuum-size=300M
df -h /
```

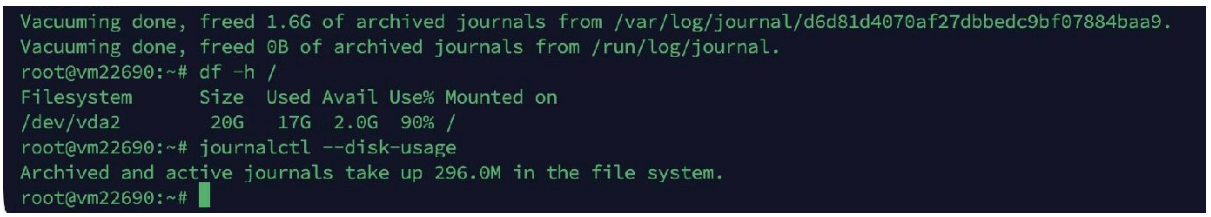


Рисунок 2. Результат очистки архивных журналов

3 Установка зависимостей

Я обновил список пакетов и установил nginx, а также проверил наличие git, curl, Python 3 и доверенных сертификатов. Node.js и npm-зависимости приложения устанавливаются внутри Docker-образа. Полное обновление ОС для выполнения лабораторной не выполнялось.

```
apt-get update
apt-get install -y --no-install-recommends \
  nginx ca-certificates curl git python3
```

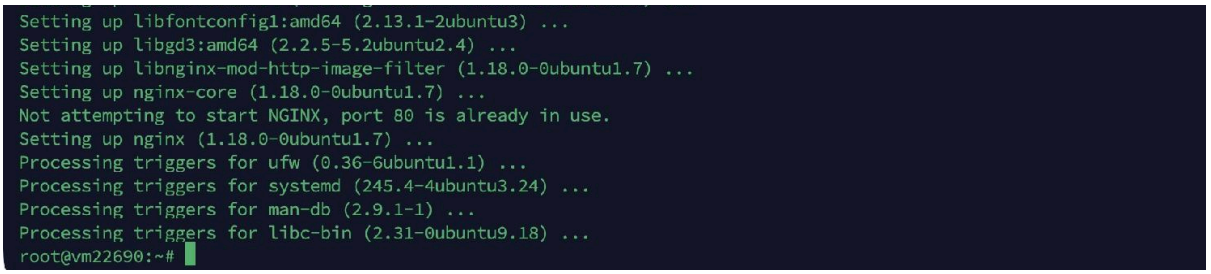


Рисунок 3. Установка пакетов nginx и зависимостей

4 Выбор портов и перенос исходников

Порт 80 уже занят работающим Caddy, поэтому для nginx выбран свободный порт 8081. Gateway принимает запросы nginx через 127.0.0.1:13000. Существующие приложения продолжили использовать свои порты.

Я клонировал ветку lab3 в /opt/rent-lab4/repository. Для ручного запуска извлёк каталоги lab2 и lab3 из коммита 7902348 в /opt/rent-lab4/manual, затем перенёс подготовленные файлы lab4.

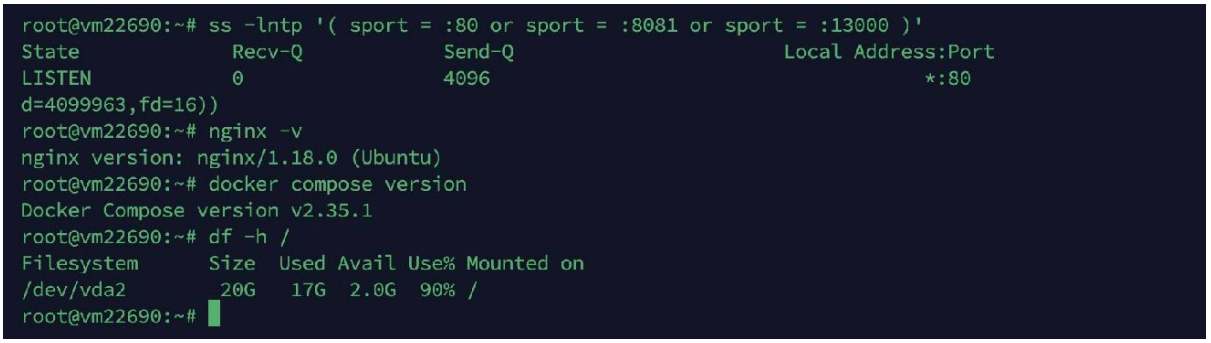


Рисунок 4. Проверка занятого порта 80 и свободных портов лабораторной

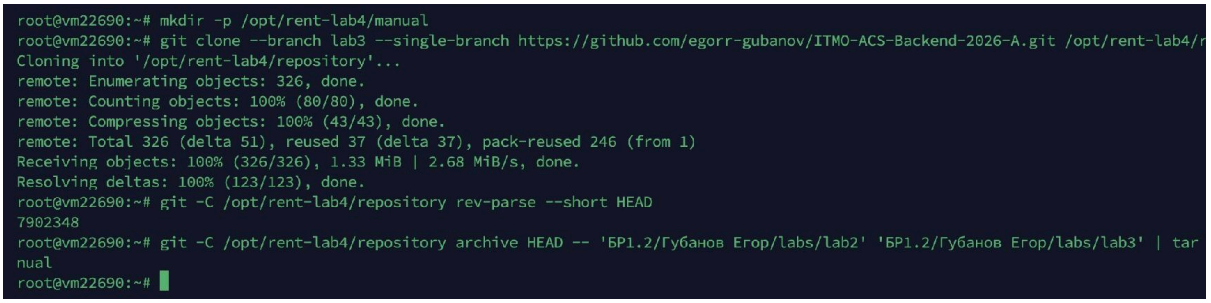


Рисунок 5. Клонирование ветки lab3 и извлечение исходного проекта

5 Серверная конфигурация приложения

Файл compose.production.yml дополняет исходный Compose из lab3. Для пяти сервисов используется один общий образ rent-lab4-app. Это позволяет повторно использовать зависимости и уменьшить расход диска. Базы и RabbitMQ доступны только в сети rent-lab4; наружу публикуется лишь локальный порт gateway.

Для JWT, межсервисных запросов, PostgreSQL и RabbitMQ я создал четыре независимых случайных секрета. Они записаны в /etc/rent-lab4/app.env с правами 0600; доступ к каталогу разрешён только root.

Для контейнеров настроены автоматический перезапуск, проверки готовности и ограничение размера журналов.

```
bash compose.sh config --quiet
bash compose.sh config --services
```

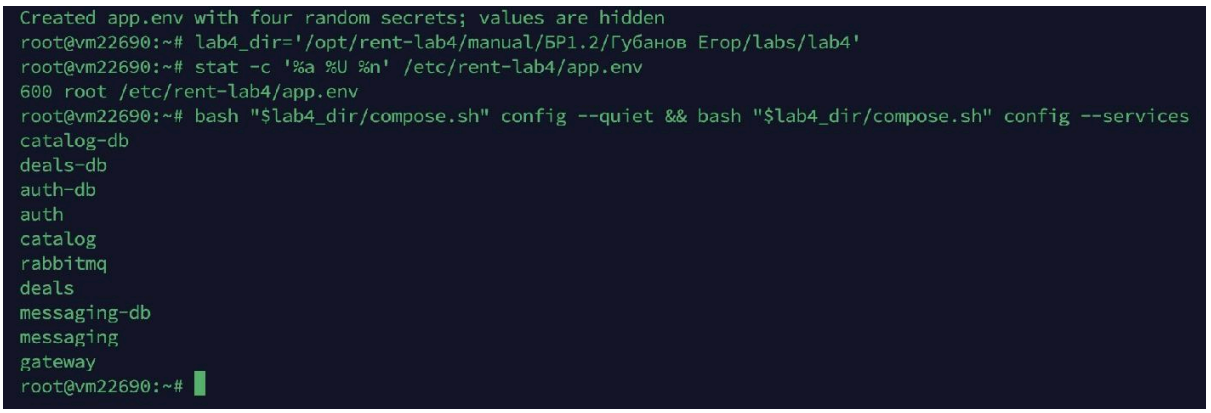


Рисунок 6. Права файла конфигурации и список сервисов Compose

6 Сборка образа и запуск контейнеров

Скрипт deploy.sh сначала проверяет конфигурацию, собирает общий образ, затем запускает контейнеры и ожидает их готовности. Данные четырёх PostgreSQL и RabbitMQ хранятся в именованных томах отдельного Compose-проекта rent-lab4. Существующие Docker-проекты не останавливались.

```
bash deploy.sh
docker ps --filter name=rent-lab4
```

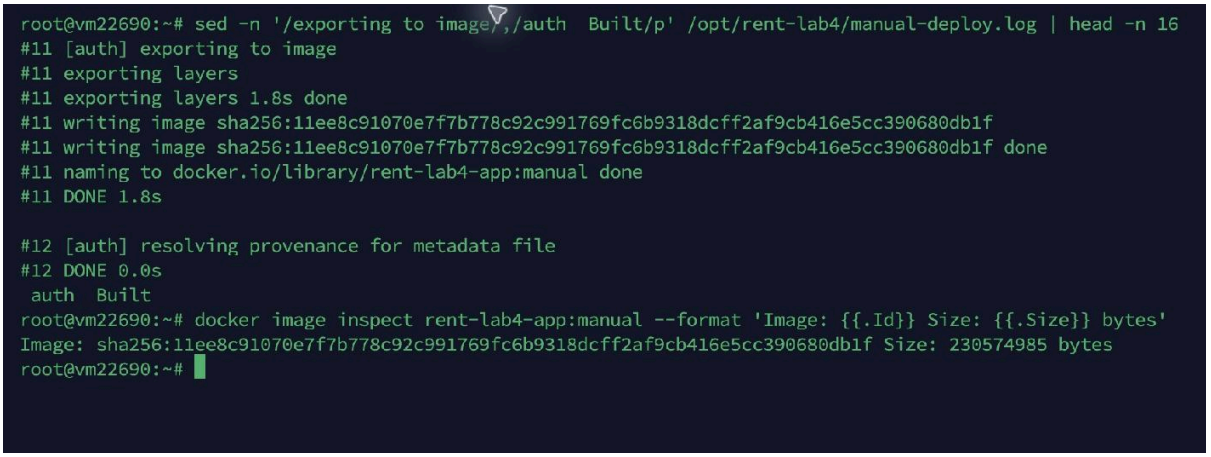


Рисунок 7. Подтверждение сборки общего образа приложения

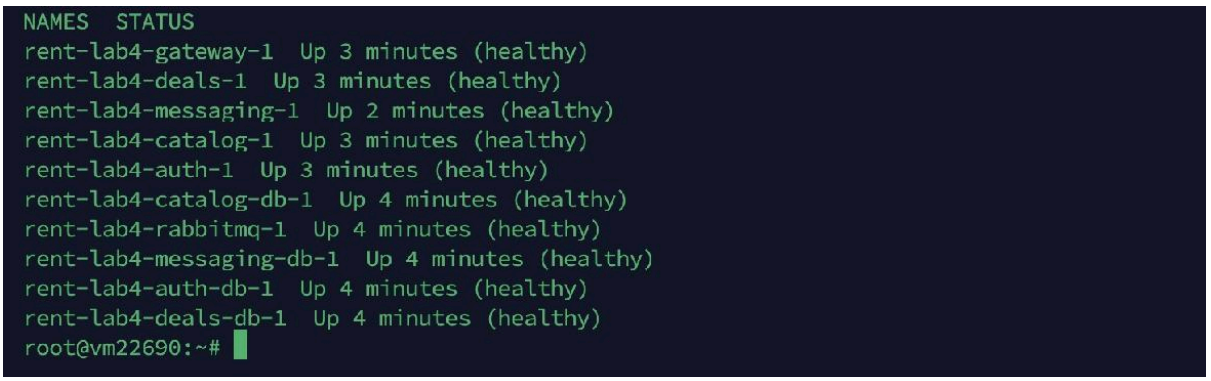


Рисунок 8. Все десять контейнеров в состоянии healthy

7 Настройка nginx и сетевого доступа

Я создал отдельный сайт nginx с listen 8081 и proxy_pass http://127.0.0.1:13000. В запросы передаются Host, X-Real-IP, X-Forwarded-For и X-Forwarded-Proto. Путь API сохраняется при проксировании.

Стандартная ссылка нового сайта nginx, использующая занятый порт 80, была отключена; исходный файл сайта сохранён. После успешной проверки nginx -t я включил автозапуск nginx. В UFW добавлено правило только для TCP-порта 8081.

```
listen 8081;
proxy_pass http://127.0.0.1:13000;
nginx -t
systemctl enable --now nginx
ufw allow 8081/tcp comment 'rent-lab4-nginx'
```

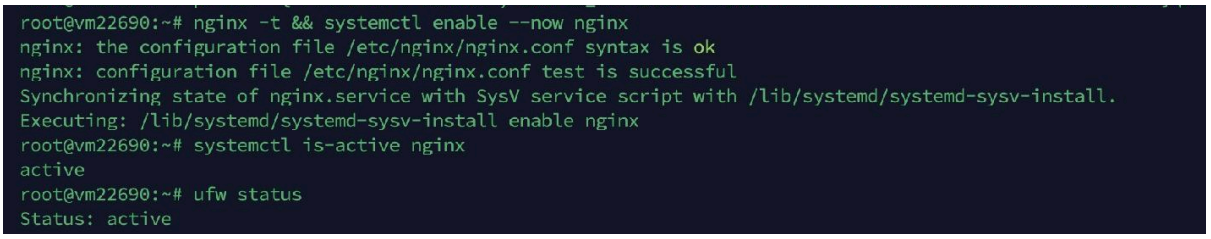


Рисунок 9. Успешная проверка nginx и активная служба



Рисунок 10. Добавление правила UFW и проверка API через nginx

8 Проверка приложения извне

По адресу http://77.239.127.10:8081/api/v1/ получен HTTP 200 и ответ rent-gateway. Запрос /api/v1/property-types/ возвращает типы недвижимости из PostgreSQL: дом, квартиру и комнату. Запрос /api/v1/me/ без токена возвращает HTTP 401. Это подтверждает доступ через nginx, работу маршрутизации и подключение каталога к базе.

Дополнительно я открыл публичный адрес справочника в браузере на локальном компьютере.



Рисунок 11. HTTP 200 и заголовок Server nginx в ответе API

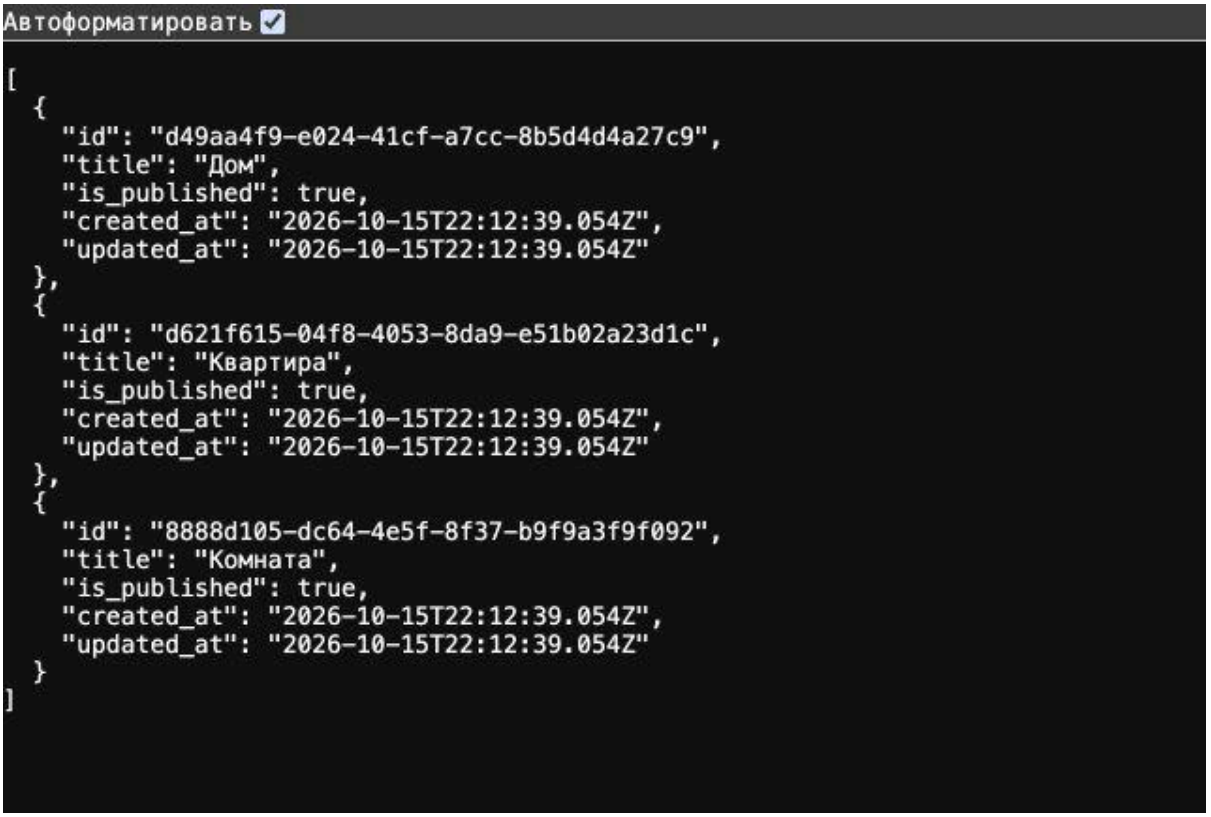


Рисунок 12. Справочник типов недвижимости по публичному адресу API

Вывод

Я подготовил удалённый сервер и вручную развернул приложение из ЛР3. Все десять контейнеров прошли проверки готовности. nginx обеспечивает внешний доступ к API через порт 8081 и направляет запросы к локальному gateway. Работоспособность проверена запросами к API на сервере и с локального компьютера.